



COURS DJANGO – IFNTI SOKODÉ

Vues, URLs, templates

03

POUR COMMENCER

Quiz sur la séance Modéliser le domaine — 15 minutes, sans machine.



Au programme

01 Le trajet d'une requête

qui écrit quoi, et dans quel ordre

03 Les routes

paramètres, noms, et jamais d'URL en dur

05 Les templates

héritage, blocs, fichiers statiques

02 Une vue par fichier

views/ en package

04 La vue

render, contexte, `get_object_or_404`

06 Compter les requêtes

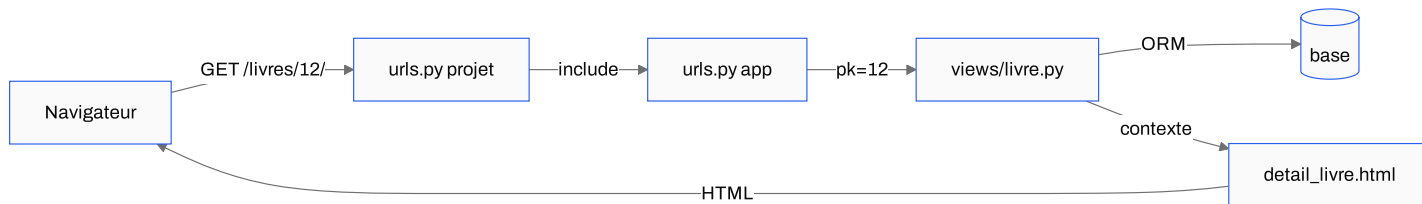
le N+1, et comment on s'en sort



01

Le trajet d'une requête

Quatre fichiers pour une page



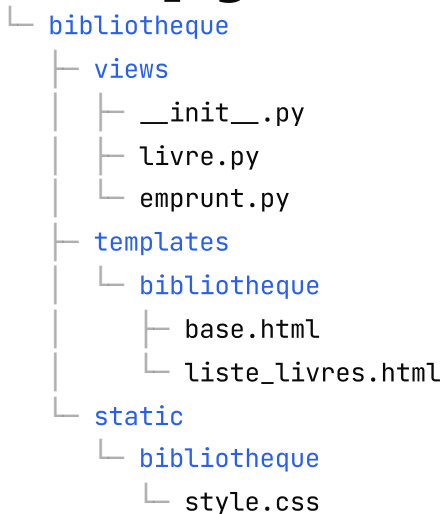
Une seule de ces étapes touche la base. Les trois autres ne font que passer.



02

Une vue par fichier

views.py devient un package



Un fichier par sujet, pas forcément par vue : `livre.py` porte la liste et le détail des livres.

`__init__.py` **réexporte**, donc `urls.py` continue d'écrire `views.liste_livres`.

Les **templates et les fichiers statiques** répètent le nom de l'application dans leur chemin, pour la même raison qu'à la séance précédente.



03

Les routes

Un motif, une vue, un nom

```
app_name = "bibliotheque"

urlpatterns = [
    path("livres/", views.liste_livres, name="liste_livres"),
    path("livres/<int:pk>/", views.detail_livre, name="detail_livre"),
]
```

- 1 **<int:pk>** — un paramètre capturé dans l'URL, converti en entier, et passé à la vue comme argument. `int` refuse ce qui n'est pas un nombre.
- 2 **name** — le nom de la route. C'est par lui qu'on la désignera partout ailleurs, jamais par son adresse.
- 3 **L'ordre compte** — Django essaie les motifs de haut en bas et s'arrête au premier qui correspond.

Ne jamais écrire une adresse en dur

DANS LE TEMPLATE	ÉCRIT EN DUR	AVEC LE NOM DE ROUTE
Le lien	/livres/12/	{% url 'bibliotheque:detail_livre' 12 %}
On renomme /livres/ en /catalogue/	tous les liens cassent, en silence	un seul fichier change
Une faute de frappe	un 404 découvert par l'utilisateur	NoReverseMatch, tout de suite



04

La vue

Récupérer, puis rendre

```
def liste_livres(request):  
    return render(request, "bibliotheque/liste_livres.html",  
                  {"livres": Livre.objects.all()})  
  
def detail_livre(request, pk):  
    livre = get_object_or_404(Livre, pk=pk)  
    return render(request, "bibliotheque/detail_livre.html",  
                  {"livre": livre})
```

- 1 **La liste** — une requête, un contexte, un template. C'est tout ce qu'une vue doit faire.
- 2 **get_object_or_404** — récupère l'objet, ou lève l'erreur 404 si le livre n'existe pas.
- 3 **Le contexte** — un dictionnaire. Ses clés deviennent les variables du template, et un nom mal orthographié ne produit aucune erreur.

CE QUE VOUS VERREZ AILLEURS

Les vues génériques

```
class ListeLivres(ListView):  
    model = Livre
```



Quatre lignes qui font le travail des huit précédentes — en cachant la requête, le nom du template et celui de la variable de contexte, qui suivent tous une convention implicite.



05

Les templates

base.html, écrit une fois

```
{% load static %}
<!DOCTYPE html>
<html lang="fr">
  <head>
    <title>{% block titre %}Bibliothèque{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'bibliotheque/style.css' %}">
  </head>
  <body>
    <nav><a href="{% url 'bibliotheque:liste_livres' %}">Livres</a></nav>
    <main>{% block contenu %}{% endblock %}</main>
  </body>
</html>
```



`{% block %}` creuse un trou que les pages filles rempliront ; ce qui est écrit entre les balises sert de valeur par défaut. `{% load static %}` est à répéter dans chaque template qui appelle `{% static %}`.

Trois lignes de plus, pas une de moins

```
{% extends "bibliotheque/base.html" %}

{% block titre %}Tous les livres{% endblock %}

{% block contenu %}
<h1>Tous les livres</h1>
{% for livre in livres %}
  <p><a href="{% url 'bibliotheque:detail_livre' livre.pk %}">{{ livre.titre }}</a></p>
{% empty %}
  <p>Aucun livre au catalogue.</p>
{% endfor %}
{% endblock %}
```



`{% extends %}` doit être la **première ligne** du fichier, et tout ce qu'une page fille écrit hors d'un bloc n'est jamais rendu — sans le moindre avertissement.



06

Compter les requêtes

SUR VOTRE PROJET, PAS SUR LA BIBLIOTHÈQUE

La liste des offres s'affiche en un clin d'œil. Elle coûte dix-neuf requêtes.

Une requête, puis deux par ligne

```
{% for offre in offres %}
<tr>
  <td>{{ offre.titre }}</td>
  <td>{{ offre.entreprise.nom }}</td>
  <td>
    {% for competence in offre.competences.all %}
      {{ competence.libelle }}
    {% endfor %}
  </td>
</tr>
{% endfor %}
```

- 1 **La vue** — une requête pour la liste des offres. Une seule.
- 2 **offre.entreprise.nom** — le nom n'est pas chargé : le template va le chercher. Une requête par offre.
- 3 **offre.competences.all** — les compétences non plus. Une requête par offre.

connection.queries

● ● ● shell Django

```
$ from django.db import connection, reset_queries
$ reset_queries()
$ for o in Offre.objects.all():
$     o.entreprise.nom; list(o.competences.all())
$ len(connection.queries)
19
$ connection.queries[-1]["sql"]
SELECT ... FROM stages_competence INNER JOIN ...
```



`connection.queries` n'est rempli que si `DEBUG = True`. En production, la liste reste vide — et le compteur ne coûte rien.

Deux outils, deux formes de relation

```
def liste_offres(request):  
    offres = (  
        Offre.objects  
        .select_related("entreprise")  
        .prefetch_related("competences")  
    )  
    return render(request, "stages/liste_offres.html", {"offres": offres})
```

- 1 **select_related** — pour une relation qui pointe vers *un* objet (ForeignKey, OneToOne). Django ajoute une jointure : toujours une seule requête.
- 2 **prefetch_related** — pour une relation qui en donne *plusieurs* (ManyToMany, relation inverse). Django fait une deuxième requête, puis rapproche les résultats en Python.
- 3 **Les deux ensemble** — ils se combinent, et c'est le cas courant.

19

- 2

MESURÉ SUR LA BASE DE
DÉMONSTRATION, NEUF OFFRES

requêtes pour la
même page, au
même code HTML
près.

MAINTENANT

Ce que vous allez faire pendant deux heures et demie

- Passer `views.py` en package
- Construire **trois pages** : la liste et le détail des offres, le détail d'une entreprise
- Les relier entre elles, **sans jamais écrire une adresse en dur**
- Écrire un gabarit commun, et une feuille de style qui tienne sur une page
- **Mesurer** le coût de votre liste, la corriger, et **remesurer**



Les trois nombres mesurés sont à rendre. Bloqué plus de **dix minutes** ? Appelez.

C'est en forgeant que l'on devient forgeron

Ce cours s'oublie, ce que vous codez reste

L'ÉNONCÉ EST SUR MOODLE – PROCHAINE SÉANCE : TESTS ET TDD